Diagnostic Report: Dispatch Packet Drafting Deviations and Systemic Resolution Strategies

1. Executive Analysis of the Systemic Anomaly

The integration of autonomous artificial intelligence agents into strict, governance-bound repository workflows relies on the absolute and deterministic application of predefined constraints. Within the Stela operational framework, the Policy of Truth (PoT.md) acts as the immutable constitution, dictating physical filing laws, epistemic standards, and execution boundaries.[1] The recent deployment cycle involving Dispatch Packet DP-OPS-0107 resulted in critical structural malformations during the drafting phase. Both tested models, operating as the Integrator (ChatGPT and Claude), failed to generate a compliant TASK.md dispatch contract, specifically faltering at the newly modified Section 3.5.1: Mandatory Closing Block.[1]

Prior iterations of the Dispatch Packet (DP) drafting protocol yielded highly consistent outputs. However, the introduction of a mandatory, pre-authored closing metadata block has exposed a fundamental architectural flaw in how the drafting prompt interacts with investigative operational tasks. This report provides an exhaustive diagnostic analysis of the failure modes exhibited by both models, synthesizes the root causes rooted in state-machine logic and prompt constraints, and formulates strategic options to restore system equilibrium. The analysis demonstrates that the observed malformations are not indicative of a regression in underlying model capability, but rather a systemic manifestation of a logical paradox within the drafting protocol itself.

2. Architectural Context and Governance Foundations

To accurately diagnose the drafting failures of DP-OPS-0107, the operational environment and its governing doctrines must be rigorously mapped. The Stela platform enforces a strict Correct-by-Construction (CbC) design discipline, which mandates structural prevention over post-execution detection.[1]

2.1 The Policy of Truth and Epistemic Axioms

The foundational laws of the repository are established in PoT.md.[1] The system operates on several core axioms that directly influence the behavior of the Integrator and Contractor agents:

- **Truth as a Primitive Predicate:** Truth is governed by specific rules rather than mere definitions. In the context of the Stela repository, "everything the system proves is actually true".[1] This dictates that documentation and metadata must strictly reflect the

mathematically verifiable state of the repository, not intent or assumption.

- **The Generation Mandate:** Verification outputs and receipts must be generated, not pre-filled or assumed. Manual fabrication of DP RESULTS receipts is explicitly prohibited.[1]
- **Anti-Drift Governance:** Any divergence between the canon and the repository state, or duplication of canon outside the Single Source of Truth (SSOT), is classified as a failure state requiring a mandatory halt (STOP WORK).[1]
- **Filing Doctrine and Context Hazards:** The repository is strictly segregated. ops/ contains executable logic, docs/ contains rationale, and storage/ contains payloads and handoffs.[1] Specifically, storage/handoff/ contains ephemeral, session-bound artifacts that represent a context hazard and must not be cited as canonical sources within immutable DP text.[1]

2.2 Staffing Protocol and the DP Workflow

The Stela governance model separates execution into distinct phases managed by specific roles:

- **The Integrator (Lead AI / System):** Maintains governance, structural integrity, and auditing. Generates Dispatch Packets and detects system drift.[1]
- **The Contractor (Guest AI):** Executes specific logic tasks and drafts implementation details within a strictly defined scope. The Contractor receives the finalized DP text and is forbidden from altering the structural boilerplate of the DP itself.[1]
- **The Operator (Human):** Owns final decisions, authorizes addenda, and executes all commits, pushes, and merges.[1]

The lifecycle of a Dispatch Packet follows an immutable workflow [1]:

1. **Open:** ./ops/bin/open establishes the freshness state.
2. **Draft:** ./ops/bin/draft generates the DP structure from the canonical template (ops/src/surfaces/dp.md.tpl). Hand-authoring structural boilerplate is forbidden.
3. **Execute:** The Contractor acts upon the authorized slot content.
4. **Certify:** ./ops/bin/certify --dp=DP-OPS-XXXX --out=auto executes receipt commands and programmatically generates the RESULTS receipt.

The integration of Section 3.5.1: Mandatory Closing Block required the Integrator to pre-author the pull request metadata, squash stubs, and technical manifests during the *Draft* phase, establishing an expectation of finality before the *Execute* phase had commenced.[1]

3. The Objective of DP-OPS-0107: CbC Phase D-1

The drafting assignment for DP-OPS-0107 centered on advancing the repository to Correct-by-Construction (CbC) Phase D-1.[1] The operational objective was to eliminate the reliance on a post-execution linter (tools/lint/leaf.sh) for telemetry enforcement and transition to structural prevention.[1]

The current linter was scored as a "B" in the Decision Registry (docs/DESIGN.md) because it acted as a "behavior-fighter"—detecting missing telemetry tags after the fact—rather than preventing the issue structurally at the moment of creation.[1]

The DP required two primary execution units:

1. **Scaffold Injection (Deterministic):** Injecting emit_binary_leaf pre-wiring into the scaffold specification (ops/lib/project/SCAFFOLD.md) so that all future generated scripts would be telemetry-wired by construction.[1]
2. **Auto-Invocation Investigation (Exploratory):** An investigative task to determine if ops/lib/scripts/common.sh could safely auto-invoke the telemetry trap when sourced, thereby eliminating the need for manual emit_binary_leaf calls in every script across the repository.[1]

This dual nature—combining a deterministic injection with a variable, outcome-dependent investigation—created an architectural conflict when subjected to the rigid constraints of the newly mandated static closing block.

4. Diagnostic Breakdown: ChatGPT Draft Malformations

The analysis of the DP-OPS-0107-CHATGPT.md draft reveals a systemic failure to resolve template variables and a complete abandonment of the Mandatory Closing Block.[1] The model successfully parsed the objective but collapsed under the contradictory requirements of investigative logic and static metadata pre-generation.

4.1 Placeholder Abandonment and Empty Blocks

Section 3.5.1 Mandatory Closing Block is designed to pre-stage the final commit and pull request metadata, including the Primary Commit Header, PR Title, PR Description, Final Squash Stub, Extended Technical Manifest, and Review Conversation Starter.[1] In the ChatGPT draft, every required field was left entirely blank, displaying only the descriptive labels (e.g., Primary Commit Header (plaintext)).[1]

This omission constitutes a major structural failure. The model recognized the schema requirements mandated by ops/src/surfaces/dp.md.tpl but failed to synthesize the conditional execution logic into definitive final-state text.[1] Because the outcome of the common.sh investigation was strictly unknown during the drafting phase, the model lacked the epistemological certainty required to write a definitive Pull Request Description or Technical Manifest, defaulting to empty placeholders to avoid violating the "Absolute Literalism" mandate.[1]

4.2 Template Variable Failures and Invented Paths

In Section 3.4.5 Receipt (Proofs to collect), the draft retained the raw template tag {{RECEIPT_EXTRA}} instead of substituting it with the scope-specific validation commands required to test the structural fixes.[1] The failure to resolve template interpolation tags indicates a breakdown in the drafting agent's ability to transition from template abstraction to concrete operational directives.

Furthermore, the Changelog (Section 3.4.3) included malformed path entries utilizing unresolved placeholder syntax [1]:

- archives/surfaces/SoP-2026-02-23-<hash>.md
- archives/surfaces/PoW-2026-02-23-<hash>.md
- archives/surfaces/TASK-DP-OPS-0107-<hash>.md
- archives/manifests/compile-2026-02-23T<timestamp>-<hash>.md

This directly violates the "Contractor Brief" rule forbidding the invention of paths or the use of globs and brace expansions.[1] The ops/bin/certify workflow generates these hashes dynamically at the exact moment of closeout.[1] By attempting to pre-register these paths in the static DP Changelog, the ChatGPT draft created an unresolvable logical loop.

4.3 Logic Looping and Contradictory Constraints

The ChatGPT draft introduced non-deterministic execution logic into the Patch / Diff section (3.4.4). It instructed the executing worker: "If safe and clean: implement... and delete tools/lint/leaf.sh... If not safe: do not implement... Keep tools/lint/leaf.sh".[1]

While logically sound for human-driven exploratory engineering, this speculative branching contradicts the Changelog manifest (3.4.3), which statically listed UPDATE tools/lint/leaf.sh without accounting for its potential deletion.[1] The DP Execution Plan is intended to be a deterministic set of steps. Including an "investigation" that fundamentally changes the final state of the repository (deleting versus retaining a core enforcement file) creates ambiguity, violating the PoT mandate against instructions capable of multiple interpretations.[1]

ChatGPT Draft Malformation	**Affected Section**	**Core Principle Violated**
Empty Metadata Fields	3.5.1 Mandatory Closing Block	DP Schema Completeness
Unresolved Tag {{RECEIPT_EXTRA}}	3.4.5 Receipt	Literalism / No Placeholders
<hash> placeholders in	3.4.3 Changelog	Path Literalism / Invented

paths		Paths
Conditional logic in implementation	3.4.4 Patch / Diff	Deterministic Execution

5. Diagnostic Breakdown: Claude Draft Malformations

The Claude draft (DP-OPS-0107-CLAUDE.md) attempted to resolve the uncertainty of the investigative task not by abandoning the fields (as ChatGPT did), but by hardcoding conditional logic into the static metadata itself. This approach resulted in severe context hazard violations, syntax malformations, and logic contradictions.[1]

5.1 Conditional Logic in Static Metadata

Unlike ChatGPT, Claude populated Section 3.5.1. However, the resulting Pull Request Description included embedded conditional statements: "If auto-invoke is viable: tools/lint/leaf.sh is deleted (scored A); total enforcement code reduced by 45 lines. If not viable: linter is retained with partial-fix notation".[1]

This is structurally malformed for a closing block. The closing metadata must represent the actual, finalized state of the repository for the ops/bin/certify execution, allowing the Operator to seamlessly copy the text into the version control system. Pre-supposing logic gates within the final commit text invalidates the integrity of the audit ledger.[1] Because execution Unit 2 results in one of two distinct, mutually exclusive outcomes, the closing metadata must reflect the *actual* result, not the algorithmic potential.

Furthermore, the Extended Technical Manifest included annotations such as (if viable; conditional) and (if tool deleted; conditional) next to file paths.[1] These structural annotations are fundamentally incompatible with automated downstream parsers that expect a strict newline-delimited list of touched files.

5.2 Context Hazard Violations and Ephemeral Data

Section 3.3 Scope and Safety explicitly defines storage/handoff/ as a directory containing session-bound artifacts that are "excluded from CDD by default" and mandates that the DP writer must not reference them.[1]

Claude's draft flagrantly violated this context hazard twice within the closing block:

1. It listed the ephemeral execution sidecar storage/handoff/CLOSING-DP-OPS-0107.md as an item in the Extended Technical Manifest.[1]
2. It instructed reviewers in the Review Conversation Starter to "Review the investigation

narrative in storage/handoff/CLOSING-DP-OPS-0107.md".[1]

Citing disposable, session-bound artifacts in the immutable DP text violates the canonical load order constraints defined in the Policy of Truth.[1] The CLOSING sidecar is an execution aid designed to be ingested by ops/bin/certify during closeout [1]; treating it as a permanent architectural artifact breaks the boundary between temporary execution state and permanent ledger state.

5.3 Command Substitution and the Certify Engine

Section 3.4.5 Receipt contains a strict operational directive: "Command substitution forms (e.g., $(pwd), $(git...)) are rejected by certify replay" and mandates the use of literal values only.[1]

Claude's draft explicitly included the following command in the mandatory receipt block:

`comm -23 <(git diff --name-only | sort) <(sort storage/dp/active/allowlist.txt) |

| true`.[1]

This utilizes process substitution (<(...)), which is interpreted similarly to command substitution by the bash shell. When ops/bin/certify attempts to replay this command using its strict literal-value engine, it will trigger a fatal parse error, causing a hard-stop during the closeout sequence.[1] This demonstrates a failure of the AI Integrator to properly weigh the specific constraint of the certify tool against its general knowledge of bash scripting best practices.

5.4 Freshness Stamp Syntax Malformation

Section 3.1 Freshness Gate explicitly defines the FRESHNESS_STAMP requirement: "YYYY-MM-DD format only. No trace tokens, no timestamps, no other text." It explicitly states that "Certify rejects all other forms".[1]

Despite this rigid constraint, the Claude draft rendered the stamp with a preceding string literal: Freshness Stamp: 2026-02-23.[1] The inclusion of the "Freshness Stamp: " text violates the "no other text" rule. The tools/lint/dp.sh schema enforcer will immediately fail the preflight check upon detecting this malformation, preventing the DP from advancing to the execution phase.

6. The Temporal Paradox of Investigative Drafting

A comparative synthesis of the failed outputs reveals that the core issue is not a degradation in Large Language Model (LLM) capabilities, nor is it a failure in context window retention. The root cause is a systemic logical paradox introduced by the prompt requirements. The system is demanding epistemological certainty regarding an event that has not yet occurred.

6.1 The Conflict of Determinism vs. Discovery

The requirement to draft the Mandatory Closing Block prior to the execution of the DP forces the AI Integrator into a temporal paradox. The Integrator is asked to accurately summarize the final state of the file system (the Extended Technical Manifest) and draft the Final Squash Stub before the Contractor has performed the exploratory work.

Drafting Constraint	Operational Reality	Resulting AI Malformation
Deterministic Changelog: The DP must declare exactly which files are marked NEW or UPDATE prior to execution.[1]	**Investigative Scope:** DP-OPS-0107 required evaluating if common.sh could safely auto-invoke without causing double-emission.[1]	**ChatGPT:** Used <hash> placeholders instead of literal paths.[1] **Claude:** Added conditional (if viable) flags to the static changelog.[1]
Mandatory Closing Block: Section 3.5.1 must contain the final PR Description and Technical Manifest at the time of drafting.[1]	**Unknown Outcome:** The result of the auto-invocation viability check (which determines if leaf.sh is deleted) is unknown during the draft phase.[1]	**ChatGPT:** Abandoned the block entirely, leaving blank labels.[1] **Claude:** Wrote "If/Then" logic into the final git commit message.[1]
Context Hazard Exclusion: Do not reference session-bound artifacts like the CLOSING sidecar in the DP text.[1]	**Reviewer Continuity:** Reviewers need to see the investigation narrative to validate the architectural decision.[1]	**Claude:** Violated the exclusion rule to point reviewers to the only place the narrative could logically exist (the sidecar).[1]

In previous operational cycles, Dispatch Packets were highly deterministic (e.g., "Normalize 35 factory and registry filenames to lowercase" [1]). For such tasks, the initial state and final state are mathematically known at the drafting phase, allowing the AI to effortlessly pre-author the Closing Block.

However, DP-OPS-0107 introduced a branching investigative pathway. The execution logs reveal exactly why the outcome was unpredictable: the auto-invocation of telemetry in common.sh was ultimately deemed *not viable* due to the constraint that the bash interpreter allows only one active EXIT trap handler per signal per shell process.[1] Because existing scripts already register their own EXIT traps for emit_binary_leaf, an auto-invoked trap in common.sh would cause a "double-emission" of the "start" leaf, violating the deterministic proof model required for the PoW.md ledger.[1] Furthermore, common.sh is sourced by intermediate library

scripts (like agent.sh), meaning an auto-invocation would poison the telemetry traces with incorrect labels.[1]

Because this technical reality was only discovered *during* the execution phase, any PR Description drafted prior to execution that assumed success (as seen in Claude's draft title: "leaf.sh scaffold injection and common.sh auto-invocation" [1]) would be factually incorrect. If the Contractor executed the DP and found the trap conflict, the pre-authored Closing Block would become a hallucination, requiring manual editing by the Operator and violating the safety invariant of "no manual edits to generated outputs".[1]

6.2 The Linter Rigidity Trap

The enforcement automation surrounding the Stela environment exacerbates the drafting failures. The system utilizes tools/lint/dp.sh to enforce the DP transaction rules and tools/lint/task.sh to enforce the container schema.[1]

The drafting instructions dictate that command substitutions are rejected by the ops/bin/certify replay engine.[1] However, to generate a dynamic comparison of allowlists and git diffs (as required by the safety invariants), standard bash paradigms rely heavily on process substitution. When the drafting AI attempts to satisfy the complex requirement of proving allowlist adherence, it naturally reaches for process substitution (comm -23 <(...)), causing a linting failure.[1]

The prompt demands a level of strict literalism that is highly susceptible to formatting slips. When an AI generates Freshness Stamp: 2026-02-23 instead of the raw string 2026-02-23, the system does not gently correct it; the linter immediately triggers a system failure state.[1] The combination of investigative uncertainty and hyper-rigid syntax validation creates a hostile environment for autonomous drafting.

7. Strategic Options: Restoring Equilibrium and Structural Integrity

To resolve the drafting instability, the architectural conflict between investigative tasks and deterministic upfront metadata must be rectified. The system cannot demand a finalized, static output block for a process that has not yet resolved its conditional pathways. The following strategic options outline pathways to realign the protocol with the capabilities of the models and the physical laws of the repository.

Option A: Typological Bifurcation of Dispatch Packets (The Protocol Split)

This approach addresses the root cause by acknowledging that not all work is strictly deterministic. It creates two distinct classifications of Dispatch Packets: Deterministic DPs and

Investigative DPs.

- **Mechanism:** When the Integrator identifies an exploratory requirement in the forward plan (e.g., "investigate common.sh viability"), the tooling mandates the generation of an Investigative DP. Investigative DPs utilize a separate template that *does not* contain a Mandatory Closing Block intended for git commits. Instead, their RESULTS output focuses solely on generating a formal Decision Record in archives/decisions/.[1] Once the decision is formalized and the constraints are known (e.g., "auto-invocation is non-viable"), a separate, standard Deterministic DP is drafted to implement the approved findings.
- **Pros:**
  - Eliminates the temporal paradox entirely. The AI Integrator is never asked to predict the outcome of a technical investigation.
  - Strictly aligns with the CbC Phase A objective of moving decision architecture out of narrative prose and into auditable, machine-readable artifacts.[1]
  - Maintains the absolute deterministic purity of the Mandatory Closing Block for implementation tasks, ensuring zero conditional logic enters the git metadata.
- **Cons:**
  - Increases the administrative overhead for the Operator, turning single exploratory tasks into two-packet cycles (Investigate -> Implement).
  - Requires modifications to the ops/src/surfaces/dp.md.tpl architecture to support multiple template typologies.
- **Risk:** Low. This strictly adheres to the existing PoT.md epistemic standard that "everything the system proves is actually true".[1] By ensuring no predictive assumptions are committed to the ledger, the system maintains absolute epistemological integrity.

Option B: Just-in-Time (JIT) Metadata Generation via Certification Toolchain Refactor

This approach modifies the system pipeline so that the Mandatory Closing Block is no longer authored during the ops/bin/draft phase, but is instead generated dynamically by the ops/bin/certify binary based on the contents of the CLOSING sidecar and the actual filesystem diff.

- **Mechanism:** Remove Section 3.5.1 from the ops/src/surfaces/dp.md.tpl drafting requirements. Instead, strictly enforce that the Contractor maintains the freeform storage/handoff/CLOSING-DP-OPS-XXXX.md sidecar continuously during execution.[1] When ./ops/bin/certify --dp="DP-OPS-XXXX" --out=auto is executed at closeout, the certify binary parses the execution sidecar, diffs the repository against the base HEAD, and programmatically renders the final Primary Commit Header, Pull Request Description, and Extended Technical Manifest.
- **Pros:**
  - Solves the placeholder abandonment seen in the ChatGPT draft; the AI is no longer tasked with filling out empty fields it cannot resolve.
  - The Extended Technical Manifest is guaranteed to be 100% accurate, as it is generated

by the binary reading the actual filesystem diff, rather than the AI attempting to predict the final paths.
    - Removes all conditional logic and context hazard citations from the static DP text.
- **Cons:**
    - Requires a material update to the ops/bin/certify binary logic and the tools/lint/dp.sh schema enforcer to handle dynamic markdown generation.
    - Requires robust parsing logic to extract the "Review Conversation Starter" and "Squash Stub" from the human/AI-maintained sidecar safely.
- **Risk:** Moderate. Enhancing the certify binary to act as an automated authoring tool for pull request metadata introduces complexity to the verification tooling. This new logic must be carefully tested against the CbC Design Discipline Preflight constraints to ensure it does not become a behavior-fighter itself.[1]

Option C: Augmented Prompt Engineering and Strict Fallback Directives

This approach attempts to fix the issue without altering the repository's toolchain or template structures, relying entirely on adjusting the initial instruction set provided to the AI Integrator.

- **Mechanism:** Update ops/lib/manifests/CONTRACTOR.md and the DP-OPS drafting prompt to include a strict "Conditional Handling Protocol." If a task involves an investigation, the AI must be instructed to author the Mandatory Closing Block based *only* on the baseline, guaranteed work (e.g., the Scaffold Injection). The prompt would strictly forbid "If/Then" logic in the closing block and mandate the use of generic placeholder phrases (e.g., "See CLOSING sidecar for investigation outcome") for the exploratory components.
- **Pros:**
    - Requires zero changes to the underlying repository binaries, linters, or template files.
    - Maintains the rapid, single-packet execution workflow.
- **Cons:**
    - Highly fragile. LLMs naturally tend toward detailed summarization, and asking them to intentionally omit critical details of an investigation from a PR summary often leads to prompt drift.
    - Does not solve the fundamental issue of the Extended Technical Manifest needing to reflect conditional file deletions (e.g., the deletion of tools/lint/leaf.sh). The manifest will either be incomplete or factually incorrect.
- **Risk:** High. The root cause of the current failure is prompt complexity exceeding the model's deterministic capabilities regarding future states. Adding more complex rules regarding conditional omissions will likely increase the cognitive load on the LLM, leading to further malformations and increased preflight failure rates.

8. Recommendation and Proposal

Based on the diagnostic evidence, the deterioration in draft quality during the DP-OPS-0107

cycle is a direct symptom of forcing deterministic static schemas onto non-deterministic investigative workflows. Attempting to patch this through further prompt engineering (Option C) will only increase the cognitive load on the LLM and exacerbate structural drift.

Recommendation: A hybrid implementation of **Option A (Typological Split)** and **Option B (JIT Metadata Generation)** is strongly recommended to permanently eliminate the temporal paradox and align the system with the Policy of Truth.

Proposed Execution Strategy

Phase 1: Implement the Typological Split (Immediate Mitigation)

1. **Halt Investigative DPs:** Institute an immediate operational directive halting the use of standard DPs for tasks requiring technical feasibility investigations.
2. **Author DP-OPS-0108 (Decision Record Architecture Finalization):** Execute a strictly deterministic packet to finalize the archives/decisions/ schema defined in CbC Phase A.[1] This establishes the formal output destination for all future investigative work.
3. **Execute DP-OPS-0109 (Investigation of common.sh):** Redraft the common.sh auto-invocation task strictly as an Investigative DP. The sole objective of this DP is to output a Decision Record indicating viability. This formally acknowledges the constraints discovered during the failed drafts (double-emission and library poisoning [1]) and writes them into the immutable ledger without requiring a static Pull Request block.

Phase 2: Refactor the Certification Toolchain (Long-term Structural Fix) 4. **Author DP-OPS-0110 (Certify Metadata Generation Refactor):** Modify the canonical template ops/src/surfaces/dp.md.tpl to remove Section 3.5.1: Mandatory Closing Block from the drafting requirements entirely. 5. **Upgrade ops/bin/certify:** Program the certify binary to ingest the execution narrative from storage/handoff/CLOSING-DP-OPS-XXXX.md and dynamically generate the final commit headers, PR description, and technical manifest at the exact moment of closeout. This perfectly aligns with the existing mandate that "Manual receipt assembly is prohibited" [1], extending that prohibition to manual PR metadata assembly. 6. **Linter Alignment:** Update tools/lint/dp.sh and tools/lint/style.sh to validate the sidecar inputs prior to certification, ensuring that the dynamic generation process receives structurally sound data.

By removing the requirement for the Integrator AI to predict the future state of the repository during the drafting phase, the system restores the deterministic integrity required by the Policy of Truth. The AI Integrator can return to flawlessly drafting sequential steps, while the execution and certification binaries assume the burden of accurately logging the resultant state, thereby eliminating the friction points observed in the DP-OPS-0107 cycle.

Works cited

1. dump-full-main-7567d65ae.txt